Foreman graph schema, v1 draft

What the graph stores and how it is linked. For your approval before we load any code into it.

Status: draft for approval. Written in plain words. The scripts that go with it were tested on a local copy of the same database engine; the one thing still to do is run them on your Docker setup (Section 6).

1. The idea in one minute

- There is **one graph per repo**. A repo is a tenant. Nothing in one graph can be seen from another.
- Inside each graph there are **three layers**:
  - **Code layer:** the map of TinyDB (files, functions, tests, and who calls what).
  - **Work layer:** what is happening now (agents, tickets, claims, messages, handoffs, results, decisions).
  - **Memory layer:** what the crew has learned (what happened, what is true, how to fix things).
- Tickets and memories both point at code. That is how a worker finds old lessons about the code it is about to change.
- Every choice the crew makes is saved as a **decision** you can click on and explain.

2. The five Track 2 focus areas and where each one lives

Focus area	Where it lives in the graph	How we show it works
Episodic, semantic and procedural memory	`Memory` things, each tagged `episodic`, `semantic` or `procedural`, linked to the code they are about	Recall shows all three kinds for one ticket. Success counts raise or lower a recipe's score. Old facts are retired, not deleted.
Persistent conversation history	`Session` and `Message` things. Messages are numbered 1, 2, 3 inside a session	30 messages written at the same instant come back as exactly 1 to 30. A restarted worker reads its own history.
Shared state and handoffs	The ticket's status in the graph, plus `Handoff` notes stored on the ticket	A stale claim reopens the ticket and the next worker finds the previous worker's note.
High-concurrency reads and writes	Every claim, message and memory update is a single database step	50 workers claim at once and exactly one wins. Full load test comes later.
Separate memory per user or tenant	One graph per repo, named `foreman_<repo>`	A secret saved for repo A is invisible from repo B.

3. The things in the graph

Code layer (loaded once from the repo)

Thing	What it stores
`File`	path
`Function`	full name (for example `tinydb.table.Table.update`), short name, `test_count` (how many tests run it)
`Test`	full test name

Work layer (changes all the time)

Thing	What it stores
`Agent`	id, role (foreman, worker, reviewer or historian), status
`Task`	run id, id, title, description, status (open, claimed, submitted, approved), who holds it, when the claim runs out, attempt count
`Session`	one conversation between an agent and a ticket; remembers its last message number

Thing	What it stores
`Message`	session id, number, who spoke, the text, the time
`Handoff`	done, remaining, tried, files touched, time
`Result`	summary, tests passed, files touched, patch reference
`Decision`	kind (claim, footprint, order, review), summary, time

Memory layer (what the crew learned)

Thing	What it stores
`Memory`	kind (episodic, semantic or procedural), summary, status (active or retired), times used, times it worked

That is 11 kinds of things. It is more than the 8 I first said, because conversation history and results each need their own.

4. The links

Link	From to	Meaning
`DEFINED_IN`	Function to File	where the function lives
`CALLS`	Function to Function	who calls whom
`IMPORTS`	File to File	planned, filled in when we load the code
`COVERS`	Test to Function	this test runs this function
`FAILS_ON`	Task to Test	the ticket's failing tests
`TOUCHES`	Task to Function	the ticket's footprint (0 hops = directly covered, 1 hop = one call away)
`CONFLICTS_WITH`	Task to Task	the two tickets share code, with a score
`HAS_SESSION`, `HAS_MESSAGE`	Agent to Session, Session to Message	conversation history
`FOR`	Session, Handoff and Result to Task	which ticket it belongs to
`WROTE`, `SUBMITTED_BY`	Agent to Handoff, Result to Agent	who wrote it
`MADE_BY`, `ABOUT`	Decision to Agent, Decision to Task	who decided what about which ticket
`ABOUT`	Memory to Function	what the memory is about
`LEARNED_FROM`	Memory to Task	where the memory came from
`SUPERSEDED_BY`	Memory to Memory	this fact was replaced by that one

5. How the rules are kept

- **Safe claim:** one database step checks the ticket is open, checks no conflicting ticket is already claimed, claims it, and records the decision. Nothing can slip in between.
- **No duplicates:** the database refuses a second `Task` with the same run and id, a second `Agent` with the same id, or two messages with the same number in one session.
- **Conflicts count rare code more:** two tickets sharing a function that only 4 tests run is a strong signal. Sharing a function that 80 tests run is a weak one. Each shared function adds `1 / test_count` to the score.
- **Memory updates over time:** each memory keeps `uses` and `successes`. Its score is `(successes + 1) / (uses + 2)`, so new memories start at a fair middle score.
- **Runs do not mix:** work-layer things carry a `run_id`. Memory is cleared at the start of each full evaluation run, except in the restart demo.

6. What was tested

All of this ran on my side, on the same engine family (FalkorDB 4.18), six times in a row with no failures:

- Setup creates 9 uniqueness rules and can be run twice safely.
- 50 workers, split across two conflicting tickets, claimed at the same instant: exactly 1 winner, exactly 1 claim decision recorded, and the other ticket blocked.
- Explain returned who decided, which ticket, which code, and which tickets were blocked.
- 30 messages written at once came back numbered 1 to 30, none lost, none repeated.
- A stale claim reopened and the handoff note was waiting for the next worker.
- All three memory kinds were recalled for a ticket, a retired fact stopped appearing, and a recipe's score went up after two successes.
- A secret in repo A was invisible from repo B.
- A duplicate agent was refused by the database.

Not yet tested: these scripts on your Docker image (you run `python demo_graph.py`), and anything on the real TinyDB data. The numbers in the example graph (such as `test_count`) are made up.

7. Changes this causes to the locked contract

The contract has no way to save conversation history. Adding it means **contract v1.1**, with four new functions:

New function	What it does
`start_session(task_id, agent_id)`	opens a conversation for a ticket
`log_message(session_id, role, content)`	saves one message, numbered safely
`get_history(session_id)`	returns the messages in order
`rate_memory(memory_id, worked)`	records whether a recalled memory helped

Everything else in v1 stays the same. Your teammate has not seen v1 either, so the whole thing goes through the change note when they are back.

8. Still open

- **Conflict threshold:** I used 0.1 on example data. It must be tuned on the real TinyDB data, where our earlier measurements showed footprints are noisy.
- **Claim timeout and heartbeat numbers:** still deferred until we measure how slow the models are.
- **Reviews:** the explain query handles claims. Review decisions point at results, so it gets one more case when we write the full set of functions.
- **Results:** the `Result` thing is defined, but its queries come with the full set of functions.